Lab 7 – Fuzz Testing with Hypothesis

Project: L6_L7_12 – Cloud Storage Tier

Name: Madan P

SRN: PES1UG24AM374

1. Objective

Use property-based fuzz testing with Hypothesis to find inputs that make the functions in processor.py raise unexpected exceptions, then fix the functions so they handle bad input safely.

- parse_storage_quota – converts strings like 500MB, 20GB, 2TB into GB.
- compute_compression_ratio – computes raw:compressed into a ratio.
- lookup_bandwidth_tier_rate – maps the first 3 letters of a tier name to a rate per GB.

2. Completed Test File

Each test feeds arbitrary text or None (st.text() | st.none()) to a function inside try/except. Any exception is raised as an AssertionError naming the function, the input and the exception type.

test_processor.py

```python
from hypothesis import given, strategies as st
from processor import parse_storage_quota, compute_compression_ratio, lookup_bandwidth_tier_
    rate


@given(st.text() | st.none())
def test_parse_storage_quota_no_crash(s):
    try:
        parse_storage_quota(s)
    except Exception as e:
        raise AssertionError(
            f"parse_storage_quota({s!r}) raised {type(e).__name__}: {e}"
        )

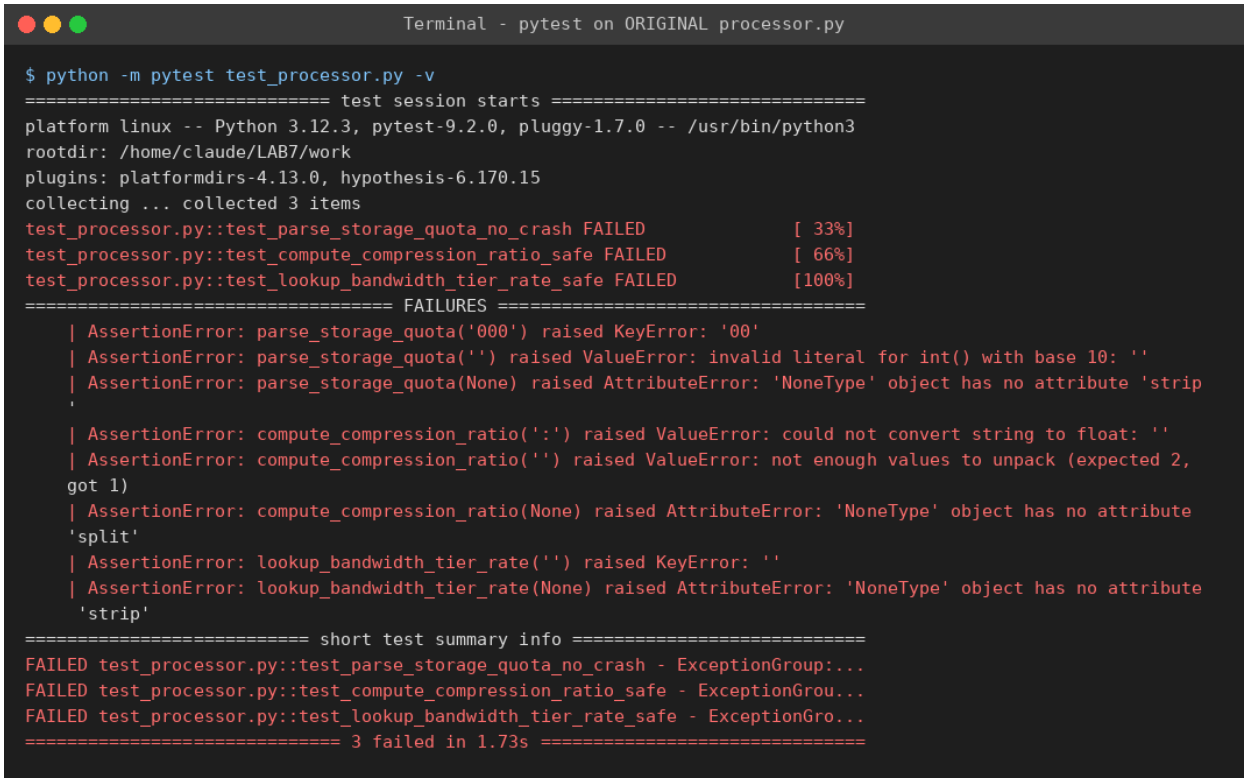

@given(st.text() | st.none())
def test_compute_compression_ratio_safe(s):
    try:
        compute_compression_ratio(s)
    except Exception as e:
        raise AssertionError(
            f"compute_compression_ratio({s!r}) raised {type(e).__name__}: {e}"
        )


@given(st.text() | st.none())
def test_lookup_bandwidth_tier_rate_safe(s):
    try:
        lookup_bandwidth_tier_rate(s)
    except Exception as e:
        raise AssertionError(
            f"lookup_bandwidth_tier_rate({s!r}) raised {type(e).__name__}: {e}"
        )
```

Screenshot 1: test_processor.py with all three tests completed

3. Test Run on Original (Buggy) Code

Running pytest against the original processor.py: all 3 tests fail, and Hypothesis reports the shrunk failing inputs.



Screenshot 2: pytest output before the fix (3 failed)

Failing inputs found by Hypothesis:

Function	Input	Exception	Cause
parse_storage_quota	''	ValueError	int() of empty string
parse_storage_quota	None	AttributeError	No type check
parse_storage_quota	'000'	KeyError	Last 2 chars taken as unit
compute_compression_ratio	''	ValueError	Unpack needs exactly 2 parts
compute_compression_ratio	':'	ValueError	float() of empty string
compute_compression_ratio	'1:0'	ZeroDivisionError	Compressed size 0 unguarded
compute_compression_ratio	None	AttributeError	No type check
lookup_bandwidth_tier_rate	''	KeyError	Prefix not in dictionary
lookup_bandwidth_tier_rate	None	AttributeError	No type check

Silent bugs found while exploring (no crash, but wrong result):

- compute_compression_ratio('nan:1') returns nan; 'inf:inf' also returns nan.
- parse_storage_quota('-5GB') accepts a negative quota.
- parse_storage_quota('1_0GB') is accepted as 10 GB because int() allows underscores.

4. Fix Applied

Each function validates its input and returns None for invalid input instead of raising. Valid inputs behave exactly as before.

- parse_storage_quota: type check plus strict regex (\d+)\s*(MB|GB|TB) using ASCII digits only.
- compute_compression_ratio: type check, exactly two parts, try/except around float(), reject nan/inf, negative raw size and compressed size <= 0.
- lookup_bandwidth_tier_rate: type check and dict.get() instead of indexing.
- main(): prints a friendly message when a function returns None.

processor.py (fixed) - parse_storage_quota

```python
import math
import re

_QUOTA_RE = re.compile(r"(\d+)\s*(MB|GB|TB)", re.ASCII)


def parse_storage_quota(quota_str):
    """
    Parses storage quota string with unit suffix into Gigabytes (GB).
    Assumes format: '<digits><2-letter unit>' (e.g. '500MB', '20GB', '2TB').
    Returns None for invalid input instead of raising.
    """
    if not isinstance(quota_str, str):
        return None
    match = _QUOTA_RE.fullmatch(quota_str.strip().upper())
    if match is None:
        return None
    value = int(match.group(1))
    unit = match.group(2)
    multipliers = {
        "MB": 0.001,
        "GB": 1.0,
        "TB": 1000.0
    }
    gb_total = value * multipliers[unit]
    return f"Quota: {gb_total:.2f} GB"


```

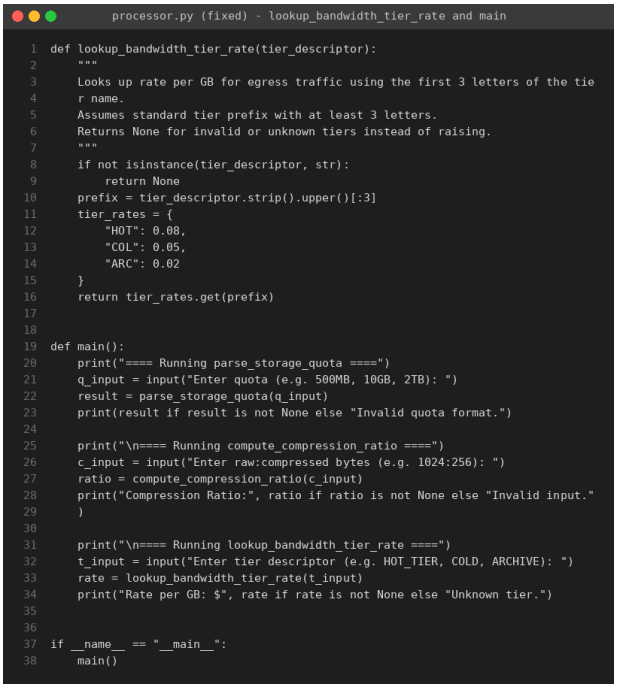
Screenshot 3a: fixed parse_storage_quota

processor.py (fixed) - compute_compression_ratio

```python
def compute_compression_ratio(size_metric_str):
    """
    Computes compression ratio: raw_bytes / compressed_bytes.
    Assumes format: 'raw_bytes:compressed_bytes' (e.g. '1024:256').
    Returns None for invalid input instead of raising.
    """
    if not isinstance(size_metric_str, str):
        return None
    parts = size_metric_str.split(':')
    if len(parts) != 2:
        return None
    try:
        raw_bytes = float(parts[0])
        comp_bytes = float(parts[1])
    except ValueError:
        return None
    if not (math.isfinite(raw_bytes) and math.isfinite(comp_bytes)):
        return None
    if raw_bytes < 0 or comp_bytes <= 0:
        return None
    ratio = raw_bytes / comp_bytes
    if not math.isfinite(ratio):
        return None
    return round(ratio, 2)


```

Screenshot 3b: fixed compute_compression_ratio




```python
def lookup_bandwidth_tier_rate(tier_descriptor):
    """
    Looks up rate per GB for egress traffic using the first 3 letters of the tie
    r name.
    Assumes standard tier prefix with at least 3 letters.
    Returns None for invalid or unknown tiers instead of raising.
    """
    if not isinstance(tier_descriptor, str):
        return None
    prefix = tier_descriptor.strip().upper()[:3]
    tier_rates = {
        "HOT": 0.08,
        "COL": 0.05,
        "ARC": 0.02
    }
    return tier_rates.get(prefix)


def main():
    print("==== Running parse_storage_quota ====")
    q_input = input("Enter quota (e.g. 500MB, 10GB, 2TB): ")
    result = parse_storage_quota(q_input)
    print(result if result is not None else "Invalid quota format.")

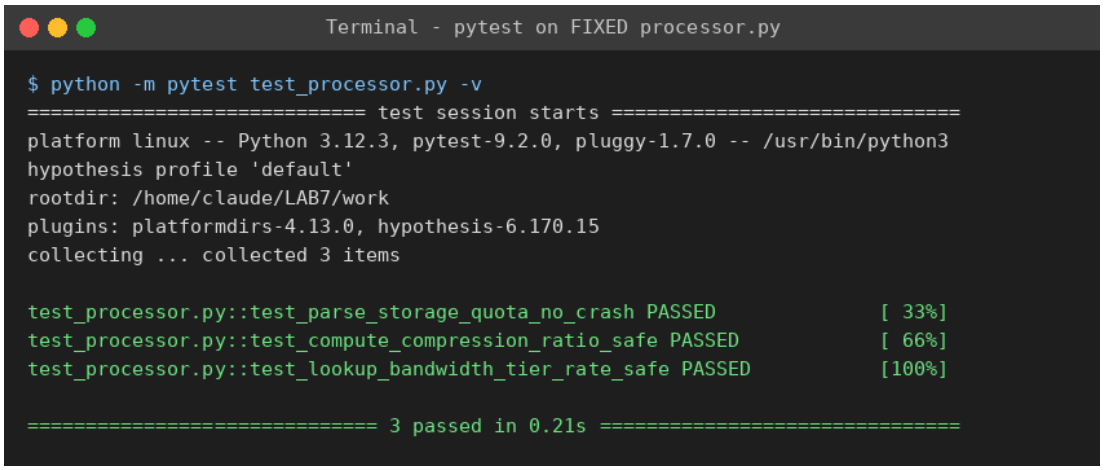
    print("\n==== Running compute_compression_ratio ====")
    c_input = input("Enter raw:compressed bytes (e.g. 1024:256): ")
    ratio = compute_compression_ratio(c_input)
    print("Compression Ratio:", ratio if ratio is not None else "Invalid input."
    )

    print("\n==== Running lookup_bandwidth_tier_rate ====")
    t_input = input("Enter tier descriptor (e.g. HOT_TIER, COLD, ARCHIVE): ")
    rate = lookup_bandwidth_tier_rate(t_input)
    print("Rate per GB: $", rate if rate is not None else "Unknown tier.")


if __name__ == "__main__":
    main()
```

Screenshot 3c: fixed lookup_bandwidth_tier_rate and main()

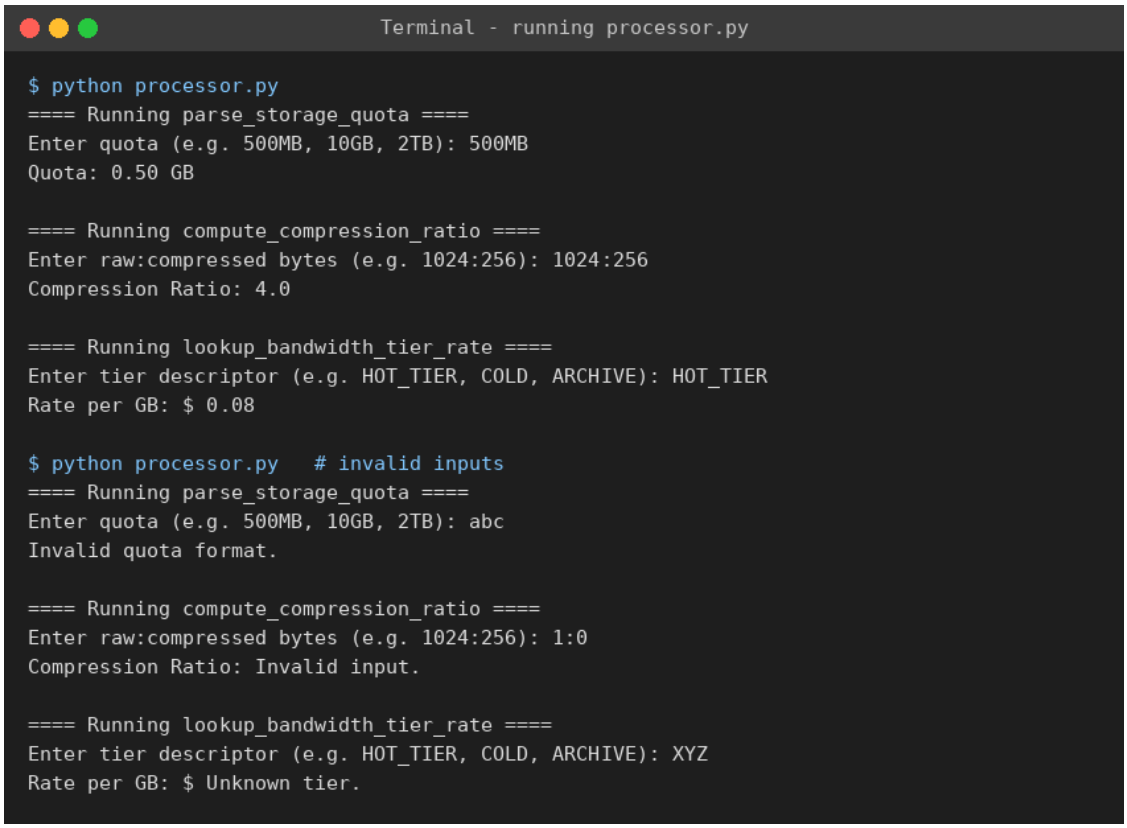
5. Test Run After the Fix



Screenshot 4: pytest output after the fix (3 passed)

6. Running the Program

Valid inputs give the same results as before; invalid inputs now give a clear message instead of a traceback.



Screenshot 5: processor.py with valid and invalid inputs

7. Conclusion

Fuzzing with Hypothesis exposed ValueError, AttributeError, KeyError and ZeroDivisionError crashes in all three functions, plus silent problems with nan, inf and negative values. After adding input validation, all three property tests pass.